



20 JUILLET 2022

APPLICATION « FPA-QUALIOPI »

[CFA NUMERIQUE, OUTIL DE GESTION DE DOCUMENTS POUR
FORMATIONS]

PASTOR GUILLAUME



Table des matières

1	Résumé Projet	2
1.1	Objectifs	3
2	Périmètre fonctionnel	4
3	Les fonctionnalités	5
4	Diagrammes Cas D'utilisations	6
5	Cahier Des Charges	9
6	Description Angular	12
7	Description LoopBack	16
8	PhpMyAdmin	18
9	Glossaire	19
10	Annexes	20
11	Veille Technologique	22

1 RESUME PROJET

Il s'agit d'un projet pour le centre de formation « CFA-Numérique ». Ce projet a été réalisé dans le cadre de mon alternance en tant que Concepteur Développeur Application, pour passer le titre professionnel de « CDA ». A la demande du responsable formation, nous devons étudier une solution qui permette de générer automatiquement des documents appelés aussi preuves lors d'audit de contrôle. Ces preuves doivent pouvoir être fournies par le responsable du centre afin d'obtenir et garder la norme « Qualiopi » afin de ne pas perdre ses subventions et ainsi que son accréditation.

« Qualiopi » est une norme de qualité pour les organismes de formations, leur donnant l'autorisation pour dispenser des cours.

L'objectif de ce projet est de créer un outil de gestion de documents pour une certification « Qualiopi », et de création de formations, avec le suivi des stagiaires.

L'administrateur de l'application, doit pouvoir créer une formation, la supprimer et même la modifier et voir les stagiaires qui la compose et pouvoir

Mais, il doit y avoir aussi la possibilité qu'un futur candidat puisse postuler à une formation en remplissant un formulaire d'inscription, déposer son CV, décrire ses compétences et expliquer ses motivations.

Et un utilisateur puisse également postuler pour devenir formateur si le centre en recherche un, en remplissant aussi un formulaire d'inscription.

1.1 OBJECTIFS

Les principaux objectifs définis de ce projet sont :

- Que le responsable du centre de formation puisse éditer en direct et devant le responsable l'organisme de contrôle les preuves demandées pour une création de formation, le nombre d'élèves, des informations sur les professeurs, le niveau de formation, l'organisation des formations, les résultats (diplômes, taux de réussite).
- Une partie est aussi à remplir par le formateur pour qu'il puisse justifier de ses diplômes, son programme et ses veilles technologiques, ses projets personnels, compte rendus de réunion ou d'informations pédagogiques.
- La possibilité d'éditer le document au format demandé et stocker sur serveur.

2 PERIMETRE FONCTIONNEL

L'outil de gestion de documents devra intégrer plusieurs traitements possibles qui sont :

- Permettre à un utilisateur qui est intéressé par une formation de pouvoir candidater en se créant un compte personnel.
- Permettre à un utilisateur de pouvoir postuler en tant que futur formateur si le centre en recherche un, celui -ci devra fournir plusieurs pièces justificatives par la suite, pour son embauche.
- Permettre au responsable du centre d'éditer lors d'un audit tous les justificatifs demandés par l'organisme « Qualiopi », créer un tableau de bord pour les élèves et formateurs, pour le suivi.

3 LES FONCTIONNALITES

Ce chapitre fait une présentation de toutes les fonctionnalités attendues pour la version « Gestion outil de documents ».

Choix techniques :

Le choix a été fait d'utiliser Angular qui est un Framework côté client, open source qui est basé sur Typescript. Ce Framework est basé sur une architecture du type MVC (Modèle-Vue-Controller), qui permet de séparer les données. Et aussi une utilisation de l'API Loopback qui servira à faire la liaison entre Angular et la base de données (phpMyAdmin). Le tout a été fait sous l'environnement Linux (Ubuntu). L'IDE utilisé est Visual Studio Code. Les échanges entre le développeur et le responsable de projet ont été faits par email, et échanges avec la plateforme Big Blue Button. Un dépôt GitLab a été créé afin de déposer le travail journalier effectué, ainsi que pour faire des sauvegardes.

```
guillaume@ubuntu:~/AngularProjet/fpa-qualiopi$ ng --version

Angular CLI
Angular CLI: 13.3.7
Node: 16.14.2
Package Manager: npm 8.12.1
OS: linux x64

Angular: 13.3.11
... animations, common, compiler, compiler-cli, core, forms
... platform-browser, platform-browser-dynamic, router

Package                                  Version
-----
@angular-devkit/architect                0.1303.7
@angular-devkit/build-angular            13.3.7
@angular-devkit/core                     13.3.7
@angular-devkit/schematics               13.3.7
@angular/cli                             13.3.7
@schematics/angular                     13.3.7
rxjs                                      7.5.5
typescript                                4.6.4

guillaume@ubuntu:~/AngularProjet/fpa-qualiopi$
```

4 DIAGRAMMES CAS D'UTILISATIONS

Voici plusieurs diagrammes de cas d'utilisation qui permettent de décrire l'interaction entre les différents acteurs concernés.

Acteurs Principaux :

- Responsable formation
- Elèves
- Formateurs

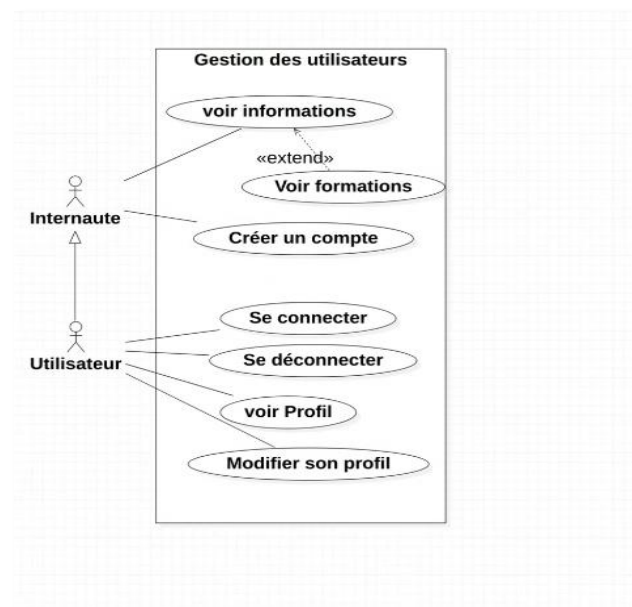


Figure 1. Diagramme cas d'utilisation futur élève.

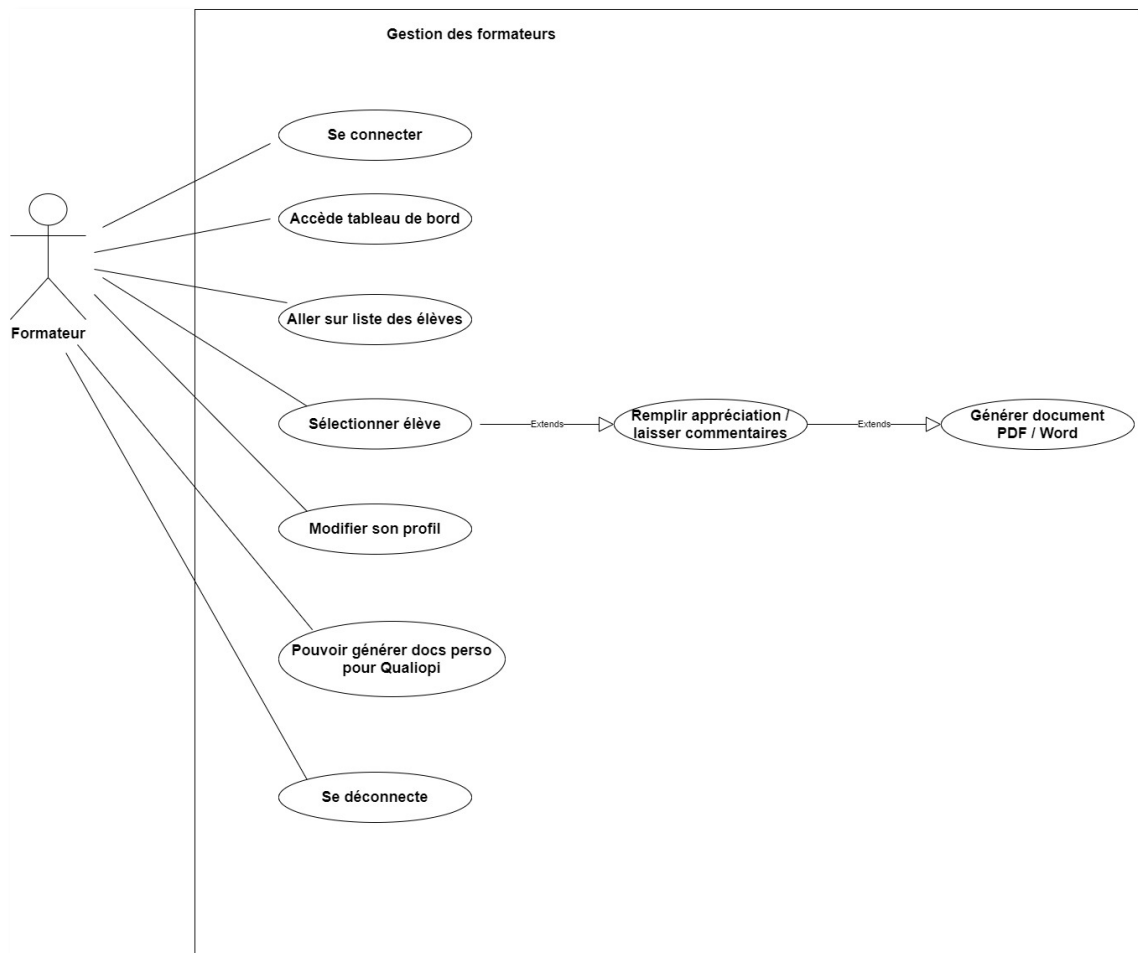


Figure 2. Cas d'utilisation pour les formateurs.

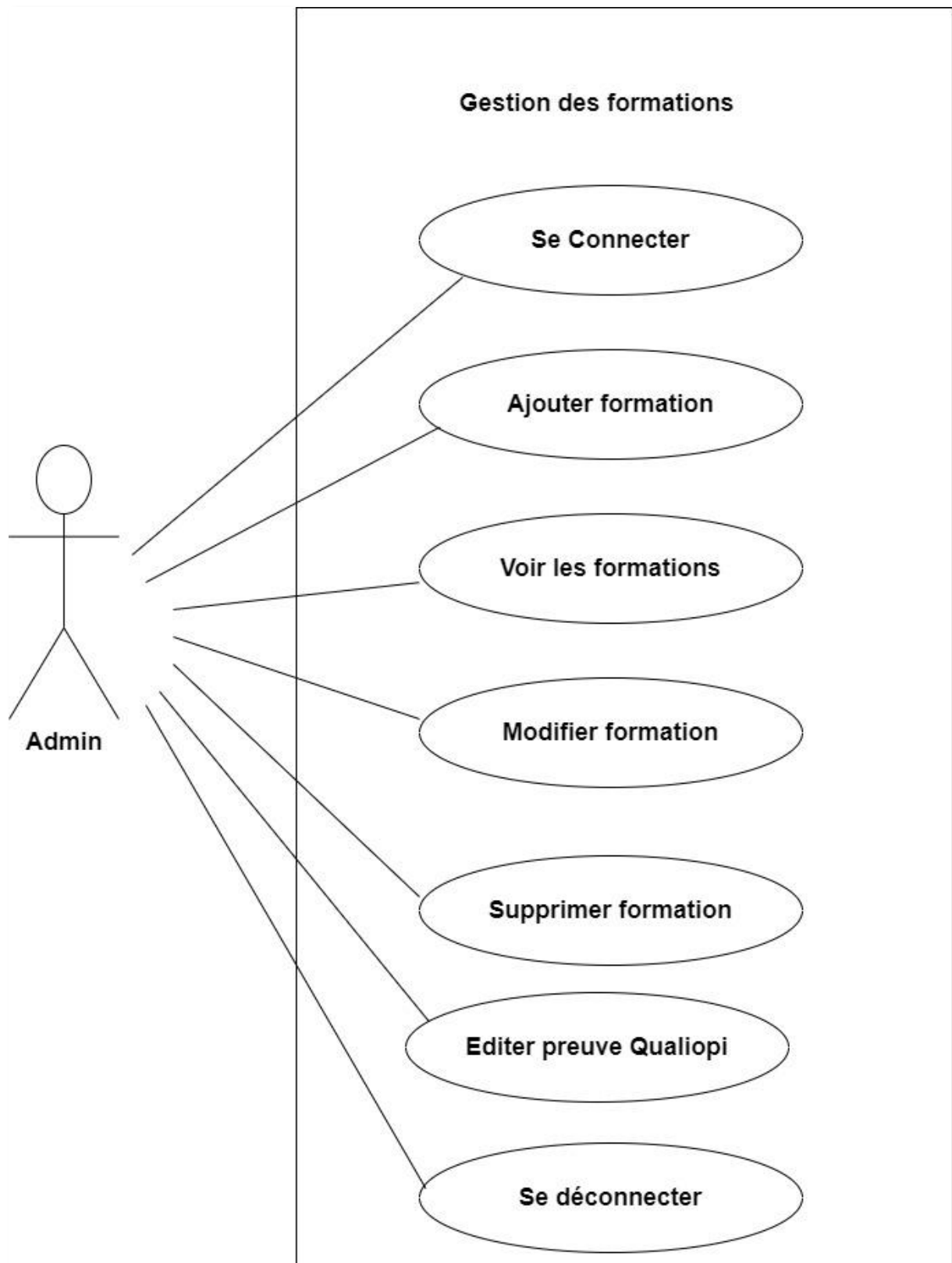


Figure 3. Gestion formation par l'admin

5 CAHIER DES CHARGES

Cahiers des Charges

But du Projet :

- Créer un ensemble d'outils d'aide à la création de documents
- Création d'un outil de gestion de preuves

Les preuves seront :

- documents (à définir)
- mise en place de la formation
- suivi de formation
- bilan de formation

Rôles

- Le système permet à un internaute possédant un compte valide de se connecter, après authentification.
- Le système permet à un internaute connecté de se déconnecter.
- Un compte peut être à zéro, un ou plusieurs rôles.
- Le rôle "administrateur" permet à l'utilisateur :
 - ajouter une formation
 - voir les formations
 - modifier les formations
 - supprimer les formations
 - créer compte
 - validé / invalidé compte
 - associer un ou plusieurs formateurs à une formation
 - dissocier un formateur d'une formation
 - administrer les preuves à chaque indicateur « qualiopi »

- Un formateur
 - peut voir ses formation associées
 - voir les élèves qui composent les formations dont il s'occupe

Données

****Compte****

- identifiant
- mot de passe
- email
- nom
- prénom
- validité ex (oui ou non)

****Formation****

- nom ex(formation développeur)
- niveau ex (niveau bac+2)
- prérequis ex (anglais A2, base algorithmie)
- organisation ex (présentiel, distanciel, mixte)

****Formateur****

- compte
- formation

****Rôles****

****Admin****

Création d'une base de données

- nom de la base : fpa-qualiopi-db
- création de plusieurs tables : formateur, formation, résultat, rôle, utilisateur
 - projet : nb étapes, nom des étapes, étape en cours, avancement par étapes
 - étapes : nb preuves, liste de preuves

6 DESCRIPTION ANGULAR

L'application fpa-qualiopi a été construite avec le Framework Angular. C'est un langage front-end est qui me semblait le plus adéquat pour faire ce travail. La CLI Angular est un outil d'interface de ligne de commande que vous utilisez pour initialiser, développer, échauder et maintenir des applications Angular directement à partir d'un shell de commande. Installez la CLI à l'aide du **npm** gestionnaire de packages :

Voici quelques commandes à connaître pour utiliser Angular sans trop de difficultés :

- Installation Angular Cli taper cette commande :

```
npm install -g @angular/cli
```

- Création d'un projet :

```
ng new nom du projet
```

Lors de la création du projet des questions seront posées à l'utilisateur créateur du projet.

Would you like to add Angular routing? No / Yes

Sélectionner Yes, car il va créer le routing automatiquement, lors de la construction du projet.

Which stylesheet format would you like to use? CSS, à choisir parmi plusieurs choix.

- Pour lancer le serveur :

```
ng serve à l'intérieur du projet
```

- Angular CLI via la commande ng serve exécute le projet sur un port par défaut (4200).

Il ne reste qu'à tester le fonctionnement dans un navigateur en lançant l'url suivante :

<http://localhost:4200>

Les composants sont le principal élément de base des applications Angular.

Un composant contrôle un morceau d'écran appelé vue.

Chaque composant se compose d' :

une classe Typescript qui définit le comportement

un sélecteur CSS qui définit la manière dont le composant est utilisé dans un modèle

un modèle (template) HTML qui déclare ce qui est rendu sur la page une feuille de styles

CSS (facultative) appliqués au modèle.

Ensuite dans le shell tape **code .** pour lancer l'IDE, sur lequel vous travaillez.

Pour créer un composant

ng generate component nomducomp

Par défaut, cette commande crée un dossier nomducomp contenant :

- Un fichier nomducomp.component.ts pour le code typescript
- Un fichier nomducomp.component.html pour la vue
- Un fichier nomducomp.component.css pour la mise en forme de la vue
- Un fichier nomducomp.component.spec.ts pour la spécification des tests

Définition d'un composant appelé aussi composant :

Un composant correspond à une classe TypeScript représentant un élément de base d'une application Angular. Une application Angular est un composant, lui-même composé d'un ensemble de composants.

Voici un tableau récapitulatifs des commandes Angular Cli

Entité	Commande	Alias
Composant	<code>ng generate component <nom du composant></code>	<code>ng g c</code>
Pipe	<code>ng generate pipe <nom du pipe></code>	<code>ng g p</code>
Directive	<code>ng generate directive <nom de la directive></code>	<code>ng g d</code>
Service	<code>ng generate service <nom du service></code>	<code>ng g s</code>
Guard	<code>ng generate guard <nom du guard></code>	<code>ng g g</code>
Module	<code>ng generate module <nom du module></code>	<code>ng g m</code>
Enum	<code>ng generate enum <nom de l'énumération></code>	<code>ng g e</code>
Class	<code>ng generate class <nom de la classe></code>	<code>ng g class</code>
Interface	<code>ng generate interface <nom de l'interface></code>	<code>ng g i</code>
Application	<code>ng generate application <nom de l'application></code>	<code>ng g app</code>

Pour plus d'informations vous pouvez aller sur le site de documentation Angular :

<https://angular.io/docs>

Cette application est constituée de plusieurs « composants », qui permettent la création des interfaces utiles à l'utilisateur pour se déplacer sur l'application. Voici la listes des différents « composants » de cette application :

- **Composant connexion** qui permettra à l'utilisateur de s'authentifier avec son email et mot de passe lorsque celui-ci voudra se connecter.
- **Composant accueil** qui expliquera à l'utilisateur les choix qui s'offre à lui, avec un bouton lui permettant de voir les formations en cours et un bouton qui lui servira pour ajouter une nouvelle formation et des onglets haut de la page.
- **Composant inscription** qui servira à l'utilisateur pour s'inscrire. Cette inscription se fera en plusieurs étapes qui sont au nombre de cinq, avec entre chaque étape une barre de chargement lui signifiant l'avancement du processus d'inscription. Le passage à l'étape suivante ne se fera que lorsque la barre de chargement

sera complète. Et il y a l'ajout d'une flèche en dessous pour rappeler à l'utilisateur où il se trouve.

- Component formation interface qui permet de visualiser toutes les formations en cours ou qui vont commencer avec un descriptif de chacune d'elles lorsqu'on clique dessus. Et enfin la création d'un bouton « supprimer formation » pour supprimer la formation qui est terminée. Elle sera effacée à l'écran est en base de données. Sur cette interface il y aura un bouton permettant de revenir à l'accueil.
- Component ajout-page-formation cette page se présente sous la forme d'un formulaire à remplir par l'utilisateur, reprenant toutes les caractéristiques pour la création d'une formation c'est-à-dire : description formation, domaine formation, prérequis formation, niveau obtenu après la formation, organisation de la formation, les objectifs, les supports de cours A la fin du formulaire se trouvera un bouton « ajouter formation » ainsi qu'un bouton pour revenir à l'accueil.

Cette application est composée aussi de « services ». Les services sont le plus souvent des singletons. Un service Angular peut être injecté par n'importe quelle classe Angular (composant, Directive, Service ou Pipe), via les paramètres de son constructeur. Pour déclarer un service Angular, il suffit de créer une classe TypeScript et de la décorer avec le décorateur @Injectable().

La bonne pratique est de toujours ajouter le décorateur @Injectable() bien que celui-ci ne soit actuellement pas obligatoire tant que le service n'a pas de dépendances.

L'utilisation d'« Observables » a été faite pendant ce projet. Les Observables sont un concept que l'on retrouve dans le Framework Angular. Il est ainsi possible de créer des services Réactifs en utilisant les Observables pour propager plus facilement les mises à jour des données dans les composants qui consomment ces données. En attendant la standardisation des Observables, Angular utilise la librairie RxJS.

Voici ci-dessous un lien expliquant la librairie RxJS.

<https://rxjs.dev/>

7 DESCRIPTION LOOPBACK

LoopBack est un Framework Node.js et Type Script primé, hautement extensible et open-source basé sur Express. Il vous permet de créer rapidement des API et des micro services composés à partir de systèmes backend tels que des bases de données et des services SOAP ou REST.

Le schéma ci-dessous montre comment LoopBack sert de pont de composition entre les demandes entrantes et les intégrations sortantes.

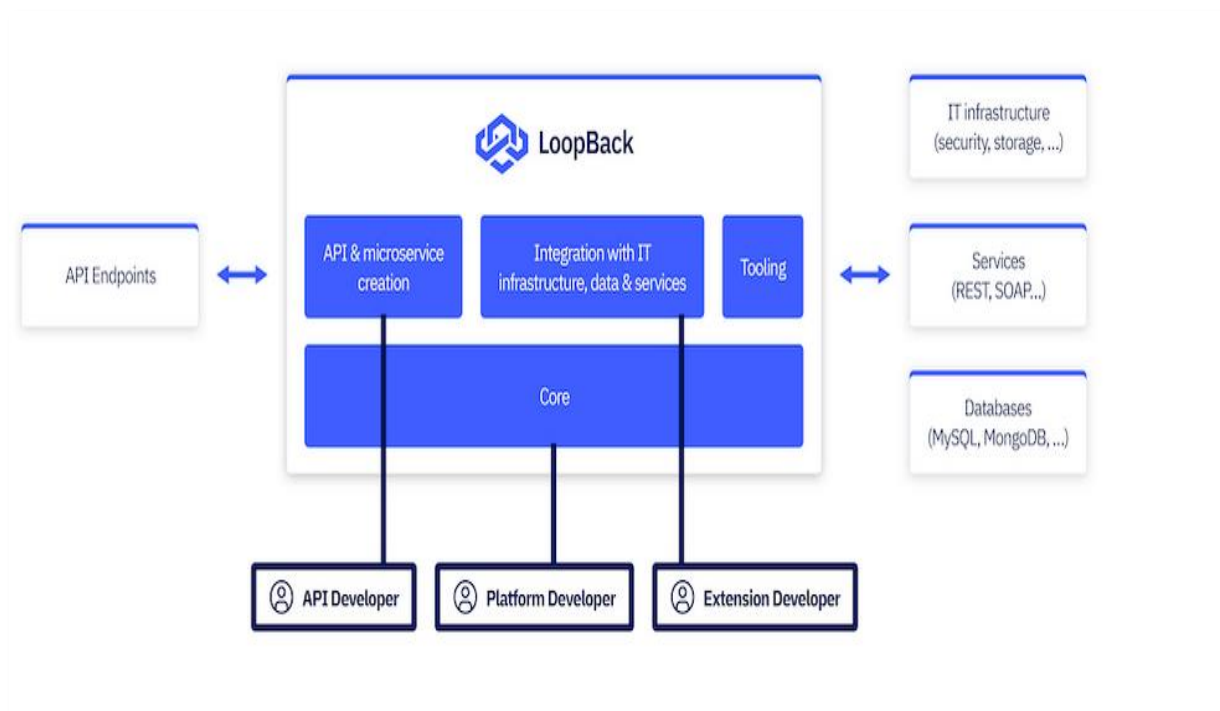


Figure 4. Diagramme Loopback

Pour cette application nous avons utilisés LoopBack 4, c'est-à-dire la dernière version en cours. Le nom choisi a été « fpa-qualiopi-api ».

Pour installer LoopBack il vous suffit d'aller sur le site officiel et de vous laisser guider par les instructions que l'on peut trouver via cette adresse :

<https://loopback.io/doc/en/lb4/>

Voici la commande à taper pour installer LoopBack

```
npm i -g @loopback/cli
```

Pour pouvoir créer un projet veuillez vous reporter à la documentation de LoopBack, lien ci-dessus et suivre les instructions, qui se trouve dans « Todo Tutorial ».

Une fois LoopBack4 installer, pour le lancer saisir la commande

```
npm start
```

Nous pouvons vérifier ce que nous venons de créer avec API Explorer.

<http://localhost:3000/explorer/>

Installer le connecteur mysql pour Loopback

```
npm install loopback-connector-mysql --save
```

Le connecteur MySQL nécessite MySQL 5.0+.

Cela installe le module à partir de npm et l'ajoute comme dépendance au fichier package.json de l'application.

8 PHPMyADMIN

phpMyAdmin (PMA) est une application Web de gestion pour les systèmes de gestion de base de données MySQL et MariaDB, réalisée principalement en PHP et distribuée sous licence GNU GPL.

Pour pouvoir enregistrer les données de l'application « fpa-qualiopi », nous avons dû créer une base de données que nous avons appelés « fpa-qualiopi-db », nous avons essayé d'être cohérent dans le nommage, entre l'application et la base. Cette base de données est composée de plusieurs tables qui sont :

- formateur
- formation
- résultat
- rôle
- utilisateur

Pour cette base, deux utilisateurs ont été créés un rôle admin et user. Le rôle admin, se connectera sur la base sous le nom admin et aura tous les droits sur cette base c'est-à-dire, il pourra ajouter un nouvel utilisateur, créer de nouvelles tables, faire des modifications à l'intérieur des tables choisies et supprimer.

Tandis que le « fpa-qualiopi-user » pourra lire et écrire dans la base de données.

Pour se connecter en tant que user sur la base de données « fpa-qualiopi-db » les identifiants se trouvent dans le projet « fpa-qualiopi-api ». Ensuite il faudra aller dans :

`src/datasources/mysql.datasource.ts`

9 GLOSSAIRE

API : (application programming interface ou « interface de programmation d'application ») est une interface logicielle qui permet de « connecter » un logiciel ou un service à un autre logiciel ou service afin d'échanger des données et des fonctionnalités.

API Rest : Une API REST (également appelée API RESTful) est une interface de programmation d'application (API ou API web) qui respecte les contraintes du style d'architecture REST et permet d'interagir avec les services web RESTful.

Angular CLI : Angular Command Line Interface (ou Interface en ligne de commande en français).

Npm : Node Package Manager, cela sert à gérer des bibliothèques Javascript. NPM est totalement écrit en JS et intégré de base dans Node. Pour l'installer, il faut exécuter la commande :

npm install

REST : Representational State Transfer.

JSON : JavaScript Object Notation. Malgré sa syntaxe très similaire à celle des objets littéraux JavaScript, JSON peut être utilisé indépendamment de ce langage et ainsi, de nombreux autres langages de programmation disposent de fonctionnalités permettant d'analyser la syntaxe du JSON et d'en générer.

Le JSON se présente sous la forme d'une chaîne de caractères , utile lorsque vous souhaitez transmettre les données sur un réseau. Il a donc besoin d'être converti en un objet JavaScript natif lorsque vous souhaitez accéder aux données.

10 ANNEXES

```
<!-- La directive *ngFor provoquera la répétition du div pour chaque élément de formation-->
<div>
  <ul class="formations">
    <li *ngFor="let formation of formations">
      <button [class.selected]="formation === selectedFormation" type="button" (click)="onSelect(formation)">
        <span class="id">{{formation.id}} &#x2011; </span>
        <span class="description"> {{formation.description}} &#x2011; </span><br>
        <span class="domaine"> {{formation.domaine}} </span>
      </button>
    </li>
  </ul>
</div>
```

Ici, j'utilise `*ngFor` d'Angular, qui est une directive structurale qui permet de boucler sur un array et d'injecter les éléments dans le DOM. On définit une méthode `onSelect()` dans `FormationsComponent` pour afficher la formation qui a été définie dans l'expression `*ngFor`.

```
<div *ngIf="selectedFormation">
  <h2>{{selectedFormation.description | uppercase}}</h2>
  <div id : {{selectedFormation.id}}</div>
  <!-- <div>domaine formation : {{selectedFormation.domaine_formation}}</div> -->
  <div>
    <label for="name">Nom de la formation : </label>
    <input id="name" [(ngModel)]="selectedFormation.description" placeholder="name" size="30">
    <br>
    <label for="domaine">Domaine de la formation : </label>
    <input id="domaine" [(ngModel)]="selectedFormation.domaine" placeholder="domaine" size="30">
    <br>
  </div>
</div>
```

`*ngIf` est une directive structurale qui permet de modifier la structure du DOM. C'est-à-dire dans ce cas lorsqu'on sélectionne une formation on affichera tous les détails la concernant.

```
<div class="boutons">
  <button (click)="onClicBoutonSuppression(selectedFormation)">Supprimer Formation</button>
</div>
```

Ici création d'un bouton pour une fonction qui supprimera une formation lorsqu'on appuie dessus. Elle se trouve dans le fichier `formations.component.html`, donc du côté vue (template).

```
onClicBoutonSuppression(formationASupprimer: Formation | undefined) {
  if (formationASupprimer === undefined)
    return ;

  this.formationService.supprimeFormation(formationASupprimer).subscribe(
    compteur => { console.log(compteur) }
  )
}
```

Ici création d'une fonction Supprimer la formation qui va sélectionné dans la base de données, lorsque je clique sur le bouton supprimer formation, se trouve dans le fichier `formations.component.ts`

```
supprimeFormation(formationASupprimer: Formation) : Observable<any>{
  return this.http.delete<Formation[]>('http://localhost:3000/formations/' + formationASupprimer.id);
}
```

Création d'un observable pour propager plus facilement la mises à jour des données dans les composants qui consomment ces datas. Avec les observables on crée des services réactifs, ce fichier se trouve `formation.service.ts`, ci en l'occurrence permet de supprimer une formation par son id.

11 VEILLE TECHNOLOGIQUE

Je présente ci-après une liste des URL des sites que j'ai consulté pour mes recherches personnelles pendant ma période de stage :

Angular :

<https://angular.io/docs>

<https://guide-angular.wishtack.io/>

LoopBack :

<https://loopback.io/doc/en/lb4/>

Librairie RxJS :

<https://rxjs.dev/>

GitLab Ldnr : Voici le lien pour récupérer les projets « fpa-qualiopi » et « fpa-qualiopi-api »

<https://gitlab.ldnr.fr/>

Fin du document